

L4-P09 大模型部署

练习手册

学习目标:从「掌握 API 调用」进阶到「独立部署 vLLM 服务并量化其性能」

本实验在 MoPaaS 平台运行时，请在创建开发环境时候，选择“vLLM”镜像。实验中的脚本不局限于用在 MoPaaS 平台。

准备工作

硬件要求

项目	最低要求	推荐配置
GPU	NVIDIA 显存 ≥ 8 GB(运行 1.5B 模型)	≥ 16 GB(支持运行 7B 模型)
系统	Ubuntu 20.04+ / WSL2	Ubuntu 22.04
Python	3.9+	3.10 或 3.11
磁盘	≥ 10 GB 可用空间	≥ 20 GB

软件安装

```
# 1. 创建虚拟环境(推荐)
python -m venv ~/llm_env
source ~/llm_env/bin/activate
()
# 2. 安装 vLLM
```

```
pip install vllm
```

3. 安装 ModelScope(用于下载模型)

```
pip install modelscope
```

4. 安装绘图与测试依赖

```
pip install pandas matplotlib aiohttp
```

5. 验证 GPU 可用

```
python -c "import torch; print('GPU OK:', torch.cuda.is_available(),  
torch.cuda.get_device_name(0))"
```

在 MoPaaS 平台, 只需要运行: `pip install pandas matplotlib aiohttp`

💡 建议提前完成此步骤, 以免实验开始时再行安装, 造成时间紧张。

文件清单

本练习包包含以下文件:

```
llm_deploy_exercises/  
├─ 练习手册.md          ← 你正在看的文件  
├─ README.md           ← 文件说明  
├─ scripts/  
│   ├── 00_download_model.sh ← 从 ModelScope 下载模型  
│   ├── 00_setup_check.sh    ← 环境自检  
│   ├── 01_serve_vllm.sh     ← 启动 vLLM 服务  
│   ├── 02_sweep_vllm.sh     ← vLLM 扫并发压测  
│   └─ 03_plot_results.py    ← 画 Rooftop 曲线  
├─ models/              ← 下载的模型会存到这里  
└─ results/             ← 你的实验输出会在这里
```

准备步骤

进入练习目录, 设置脚本可执行权限, 然后依次完成环境自检与模型下载:

```
cd llm_deploy_exercises  
chmod +x scripts/*.sh scripts/*.py
```

```
# 1. 环境自检(检查 GPU、Python、vLLM、依赖库、端口、磁盘是否就绪)
bash scripts/00_setup_check.sh

# 2. 下载模型(默认 Qwen2.5-1.5B-Instruct,约 3 GB,支持断点续传)
bash scripts/00_download_model.sh
```

下载脚本内部仅包含一行命令,等效于:

```
modelscope download --model Qwen/Qwen2.5-1.5B-Instruct --
local_dir ./models/Qwen2.5-1.5B-Instruct
```

完成后,模型位于 `./models/Qwen2.5-1.5B-Instruct/`。

关于模型选择的说明

本练习默认使用 **Qwen2.5-1.5B-Instruct**(约 3 GB,8 GB 显存即可运行),理由:

- 启动迅速(约 15 秒),单轮压测仅需数分钟,便于反复调整参数
- 在 RTX 4090 上可支持 100+ 并发,**Rooftop 曲线变化幅度更大,拐点更易识别**
- 任何具备 GPU 的笔记本均可运行,便于课后继续实验
- **所有结论同样适用于 7B / 14B / 70B 模型**,公式不变,仅数值差异

如需使用 7B 模型(显存 ≥ 16 GB),请同步修改以下三处:

1. `scripts/00_download_model.sh`:`--model` 参数改为 `Qwen/Qwen2.5-7B-Instruct`,`--local_dir` 改为 `./models/Qwen2.5-7B-Instruct`
2. `scripts/01_serve_vllm.sh`:`MODEL_ID` 默认值改为 `Qwen/Qwen2.5-7B-Instruct`
3. `scripts/02_sweep_vllm.sh`:`MODEL_PATH` 改为 `./models/Qwen2.5-7B-Instruct`

实验结论相同,仅绝对数值有差异。

练习:vLLM 部署与压力测试

学习目标

1. 启动 vLLM 服务,通过 OpenAI 兼容接口调用模型
2. 完成一次并发扫描压测,获取 TTFT、TPOT、Throughput 三组数据
3. 绘制 Rooftop 曲线,确定本机硬件的「黄金运营点」
4. 解读 vLLM 启动日志,反推 KV Cache 池实际大小

步骤 1 • 启动 vLLM 服务

操作

```
bash scripts/01_serve_vllm.sh
```

脚本内部实际执行的命令:

```
vllm serve ./models/Qwen2.5-1.5B-Instruct \
  --served-model-name ./models/Qwen2.5-1.5B-Instruct \
  --max-model-len 8192 \
  --enable-prefix-caching \
  --gpu-memory-utilization 0.9 \
  --host 0.0.0.0 --port 8000
```

💡 `--served-model-name` 同样设为本地路径,使后续的 `curl` 和 `vllm bench` 使用同一字符串,避免任何网络下载。

解读启动日志

服务启动后,关注日志中的两行关键信息:

```
INFO ... # GPU blocks: ____ ← KV Cache 池大小
INFO ... Maximum concurrency for 8192 tokens per request: ____x
```

💡 建议进行心算验证:使用模块一学习的显存预算公式估算理论 KV 池大小,然后与日志中的实际值对比,两者差值即为运行开销 R 的实际大小。这种「理论 vs 实测」的对照训练,有助于建立

部署工程的直觉。

测试 API 可用性

新开一个终端,执行:

```
curl http://localhost:8000/v1/chat/completions \
-H "Content-Type: application/json" \
-d '{
  "model": "./models/Qwen2.5-1.5B-Instruct",
  "messages": [{"role": "user", "content": "你好,请用一句话介绍自己"}]
}'
```

💡 *model 字段使用本地路径,需与 vLLM 启动时的 --served-model-name 保持一致。*

收到结构合理的 JSON 响应即表明服务运行正常。

步骤 2 • 并发扫描压测

操作

另开一个终端(保持 vLLM 服务运行):

```
bash scripts/02_sweep_vllm.sh
```

脚本将依次执行并发 $1 \rightarrow 2 \rightarrow 4 \rightarrow 8 \rightarrow 16 \rightarrow 32 \rightarrow 64 \rightarrow 128$ 共 8 档压测,每档发送足够数量的请求以确保统计稳定。

运行期间的输出

每档并发开始前,脚本会先打印即将执行的命令,然后输出 vllm bench 的进度,示例:

► 测试并发数 C=8(160 个请求)

即将执行:

```
vllm bench serve --backend openai \
  --model ./models/Qwen2.5-1.5B-Instruct \
  --base-url http://localhost:8000 \
  ...
```

[vllm bench 的进度输出]

✓ 完成,耗时 23 秒,结果保存至 results/c8.json

扫描期间的延伸思考

- 查看 results/c1.json 的字段结构: `cat results/c1.json | python -m json.tool | head -40`
- 思考:为什么 num-prompts 需要随并发数增加?*(提示:并发越高,需要更多样本才能保证统计稳定性)*

完成后的预期产出

results/ 目录下应包含 8 个 JSON 文件:

```
ls results/  
# c1.json c2.json c4.json c8.json c16.json c32.json c64.json c128.json
```

步骤 3 • 绘图与拐点分析

操作

```
python scripts/03_plot_results.py
```

产出

会生成两个文件:

- results/summary.csv — 指标汇总表
- results/rooftop.png — 三张图组合

summary.csv 的格式

```
concurrency,mean_ttft_ms,p95_ttft_ms,mean_tpot_ms,p95_tpot_ms,output_throughput  
,request_throughput  
1,118,135,9.01,10.2,202,1.58  
2,124,142,9.03,10.5,402,3.15  
...
```

💡 不同 vLLM 版本的字段名可能略有差异。若 P95 列全为 0,说明当前版本未输出 P95 字

段:03_plot_results.py 会自动 fallback 到 mean 值,不影响曲线形状。

rooftop.png 的三张子图

子图	内容	关注重点
左	吞吐 vs 并发	曲线从何处开始饱和(变平)
中	TPOT vs 并发	拐点位置(数值在何处突然上升)
右	Rooftop:吞吐 vs TPOT	黄金运营点(曲线右上角的转折点之前)

拐点分析

观察自己绘制的图表,回答以下问题:

1. 本机的拐点出现在哪个并发数?
2. 若业务要求 P95 TPOT < 50ms,可支持的最大并发数是多少?
3. 若业务可容忍 P95 TPOT 100ms,并发上限可提升至多少?
4. 上述并发数对应的「业务用户数」分别是多少?*(客服场景 think time $\approx 10x$, Agent 场景 $\approx 1x$)*

以上问题没有标准答案,实际数值受硬件、模型、参数等因素影响。**关键在于具备「基于数据进行分析」的能力**,这是部署工程师的核心素养。

步骤 4 • 延伸实验(可选)

难度	实验内容
★	将 02_sweep_vllm.sh 中的 INPUT_LEN 改为 4096,重新运行,对比拐点位置的变化
★★	使用 ENABLE_PREFIX_CACHE=0 bash scripts/01_serve_vllm.sh 关闭 prefix caching 后重新压测,观察 TTFT 的差异
★★	显存 ≥ 16 GB 时,将下载脚本中的模型替换为 Qwen2.5-7B-Instruct,观察大模型对并发能力的影响
★★★	改用 INT4 量化版 Qwen2.5-7B-Instruct-AWQ,对比 7B 模型量化前后的并发能力

常见问题排查

Q1:启动 vLLM 时报「CUDA out of memory」

原因:显存不足。

解决方法(依次尝试):

1. 将 01_serve_vllm.sh 中的 MAX_LEN 调小至 4096 或 2048
2. 将 GPU_UTIL 调整为 0.85
3. 改用更小的模型:将下载脚本和启动脚本中的模型名均改为 Qwen/Qwen2.5-0.5B-Instruct

Q2:端口 8000 被占用

```
# 查看占用进程
lsof -i:8000
# 终止进程(替换为实际 PID)
kill -9 <PID>
# 或更换端口启动:PORT=8001 bash scripts/01_serve_vllm.sh
```

⚠ 若更换端口,需同步修改 scripts/02_sweep_vllm.sh 顶部的 URL(该变量目前硬编码为 8000)。

Q3:vllm bench 命令不存在

vLLM 版本过低(bench 子命令在 0.6.4+ 版本引入)。检查并升级:

```
pip show vllm | grep Version    # 应 >= 0.6.4
pip install -U vllm
```

Q4:压测结果异常(P95 数值过大)

通常因缺少 warmup 导致。首批请求处于冷启动阶段,延迟显著偏高。02_sweep_vllm.sh 已内置 warmup 步骤;若自行编写脚本,需注意补充。

Q5:绘图脚本输出中文乱码

需安装中文字体或在脚本中显式指定:

```
import matplotlib
matplotlib.rcParams['font.sans-serif'] = ['SimHei', 'WenQuanYi Zen Hei',
'PingFang SC']
matplotlib.rcParams['axes.unicode_minus'] = False
```

03_plot_results.py 已包含该配置。如仍出现乱码,请确认 `~/fonts/` 目录中是否存在中文字体。

完成练习后

应有的输出文件

```
results/
├─ c1.json           # 并发 1 压测结果
├─ c2.json
├─ c4.json
├─ c8.json
├─ c16.json
├─ c32.json
├─ c64.json
├─ c128.json
├─ summary.csv      # 汇总表
└─ rooftop.png      # 三张性能图
```

这些文件并非需要提交的作业,而是**个人工程参考资料**。日后进行类似的部署评估时,可直接复用本套脚本和数据。

自检问题

完成练习后,尝试回答以下问题:

- 本机 Qwen2.5-1.5B 在 vLLM 下的并发拐点大致位于何处?
- KV Cache 池的实际大小与公式计算的理论值,差距有多大?
- 业务要求 P95 TPOT < 50ms 时,可支持的最大并发是多少?

- 业务要求 TTFT < 500ms 时,并发上限又是多少?
- 若改用 7B 模型,预测拐点会移动到何处?

能够基于数据准确回答以上问题,即表明已掌握部署工程师的核心思维方式。

结语

「测不出来的优化等于没优化。」

本练习的核心价值,不在于得出本机的具体性能数字,而在于建立基于数据进行工程决策的方法论。

工作中遇到类似的「如何选型、如何配置」问题时,本套脚本经少量调整即可复用。